

**Master 1 Informatique****Programmation Orientée Objet Avancée en Java****TD /TP N° 2 : Classes, objets, héritage et exceptions en Java****Exercice 1 : Simulation d'une carte téléphonique**

Une carte téléphonique de 50 unités peut être représentée sous forme d'un objet avec les messages suivants :

- **debiterUnites (debit)** : débite une carte d'un nombre variable d'unités **debit** ;
- **getUnites** : renvoie le nombre d'unités restantes d'une carte.

Écrire une classe Java qui simule une carte téléphonique.

**Exercice 2 : Comptes bancaires**

Un compte bancaire peut être représenté sous forme d'un objet ayant trois attributs privés **numeroCompte**, **titulaireCompte** et **credit**.

1. Écrire une classe Java **CompteBancaire** permettant de représenter un compte bancaire ayant, en plus du constructeur permettant l'initialisation des valeurs d'attributs et des méthodes get et set, la méthode suivante :
  - **debiter (debit)** : débite le compte d'un montant **debit** ;
2. Écrire une classe **Banque** qui permet d'utiliser la classe **CompteBancaire**.

Le numéro, le titulaire du compte et le montant du crédit seront envoyés comme arguments sur la ligne de commande.

Pour tester la méthode debiter(), la valeur du montant à débiter sera saisie en utilisant la classe **java.util.Scanner** ou la classe **javax.swing.JOptionPane**.

Après avoir débité le compte, il faudra afficher sous la forme :

*« Le compte numéro XXX appartenant à YYY a un crédit de ZZZ. »*

3. Écrire une classe **CompteEpargne** qui hérite de la classe **CompteBancaire** et qui a la particularité d'avoir un attribut **tauxEpargne**.

**Exercice 3 : Représentation de points et segments**

1. Écrire une classe Java **Point** permettant de décrire les coordonnées d'un point dans le plan. Cette classe a :
  - des attributs privés x et y.
  - un constructeur **public Point (int x, int y)** qui permet d'initialiser le point lors de sa

création.

- des méthodes **public int getX()** et **public int getY()** qui retournent respectivement les valeurs de x et de y.
  - une méthode **public String toString()** (appelée par `println()`) qui retourne une chaîne de caractères de la forme "(1,4)".
  - une méthode de test **public static void main(String[] args)** qui crée deux points et affiche leurs coordonnées.
2. Écrire une classe Java **Segment** permettant de décrire un segment dans le plan. Celui-ci est défini par deux points : ses extrémités. Cette classe a :
- des attributs de type `Point` : `p1` et `p2`.
  - un constructeur **public Segment(Point p1, Point p2)** qui permet d'initialiser le segment lors de sa création.
  - une méthode **public String toString()** (appelée par `println()`) qui retourne une chaîne de caractères de la forme "[(1,4)..(2,5)]".
  - une méthode de test **public static void main(String[] args)** qui crée d'abord deux points, à partir de coordonnées passées sur la ligne de commande, et crée ensuite un segment à partir de ces points.

#### Exercice 4 : Représentation d'un cercle

Écrire une classe `Cercle` étendant la classe `Point` de l'exercice 3 précédent.

Cette classe a pour :

- attributs **private double rayon** et **private Point centre**
- deux constructeurs : **public Cercle () {...}** et **public Cercle (Point centre, double rayon) {...}**
- des accesseurs : **public double getRayon(){...}** ; **public setRayon(double rayon){...}** ; **public Point getCentre(){...}** et **public setCentre(Point centre){...}**
- la méthode **public double aire()**
- la méthode **public String toString()**

une méthode de test **public static void main(String[] args)** qui crée d'abord un point, à partir de coordonnées passées sur la ligne de commande, et crée ensuite un cercle de centre ce point.

#### Exercice 5: Formation

On a conçu les deux classes suivantes permettant de représenter des enseignants et des formations. On suppose que chaque formation possède un seul responsable, et qu'un professeur ne peut pas être responsable de plus d'une formation.

```
public class Prof {  
    private String nom, prenom;  
    public Prof (String nom,String prenom){  
        this.nom=nom;  
        this.prenom=prenom;  
    }  
    public String getNom(){  
        return nom;  
    }  
}
```

```

        public String getPrenom(){
            return prenom;
        }
    }
    public class Formation{
        private String nom;
        private Prof responsable;
        public Formation(String nom, Prof p){
            this.nom=nom;
            this.responsable=p;
        }
        public String getNom(){
            return nom;
        }
        public Prof getResponsable(){
            return responsable;
        }
    }

```

1. Modifier ces classes de sorte que la classe Prof propose une méthode getFormation qui renvoie la formation dont l'enseignant est responsable.
2. Modifier ces classes de sorte que l'on puisse changer le responsable d'une formation.

### **Exercice 6 : Matériel informatique**

On se propose de développer une application pour un magasin spécialisé dans la vente de matériels informatiques : ordinateurs (PCs et portables) et périphériques (imprimantes, scanners).

Tout produit du magasin est caractérisé par une référence, une marque, une date de fabrication et un prix.

Un ordinateur est considéré comme un produit caractérisé par son processeur (ex : Intel Dual Core T2370 à 1,73 GHz), la taille de sa RAM et du disque dur exprimées en Giga-octets et le système d'exploitation avec lequel il est livré. Pour les ordinateurs de bureau (PC), on a l'information sur la carte graphique alors pour les ordinateurs portables, on a les caractéristiques concernant l'écran (type et taille).

Les imprimantes sont caractérisées par leur type (laser, jet-d'encre, ...), leur résolution et leur vitesse d'impression. Les scanners sont caractérisés par leur résolution.

1. Écrire une classe **Produit\_Informatique** permettant de représenter les équipements vendus par le magasin. La classe devra comporter au moins un constructeur et des méthodes de type *get* et *set*.
2. Écrire des classes qui héritent de la classe **Produit\_Informatique** et permettent de représenter les ordinateurs et les périphériques. Chaque classe devra comporter au moins un constructeur et des méthodes de type *get* et *set*.
3. Écrire une classe **Test** qui contiendra une méthode *main* permettant de créer et de manipuler des instances des classes créées précédemment.

### **Exercice 7 : Représentation et Manipulation de Polynômes**

Un polynôme (par exemple  $2-3x^2+x^4$ ) peut être représenté par un tableau comme suit :

| 0 | 1 | 2  | 3 | 4 |
|---|---|----|---|---|
| 2 | 0 | -3 | 0 | 1 |

1) En adoptant cette représentation, définir une classe Java nommée **Polynome** permettant de réaliser les opérations suivantes :

- construction d'un polynôme à partir d'une notation vectorielle (par exemple 2, 0, -3, 0, 1 pour le polynôme précédant)
- addition de deux polynômes
- affichage d'un polynôme
- dérivée d'un polynôme
- calcul de la valeur d'un polynôme pour un réel donné.

2) Écrivez un programme principal :

- saisissant un polynôme P
- calculant et affichant  $P + 2 - 3x^2 + x^4$
- calculant et affichant la dérivée de P en 2

3) Construire une classe **Polynome2** adaptée aux polynômes de degré 2 à partir de la classe **Polynome**. Cette classe doit posséder une méthode nommée **ResoudreEquation** cherchant les solutions de l'équation du second degré associée au polynôme.

#### **Exercice 8 : Moyenne de nombres**

Écrire un programme **Java** qui à partir d'un tableau d'entiers représentés en chaînes de caractères (String[] liste) calcule la moyenne de ces nombres. Le programme doit traiter le cas où une chaîne de caractères ne correspond pas à un entier en provoquant une exception. Le programme doit aussi traiter le cas où l'entier n'est pas compris dans un intervalle donné [min, max].

#### **Exercice 9 : Utilisateurs**

On veut développer une classe Java, nommée *User*, permettant de représenter des utilisateurs d'un système. Chaque utilisateur est caractérisé par un *login*, un *mot de passe* (chaîne de 11 caractères exactement), un *nom* et un *prénom*.

Écrire la classe *User* en fournissant des méthodes *get* et *set* pour les attributs. Le constructeur doit permettre de créer un utilisateur à partir des informations concernant son login, son nom et son prénom. Un mot de passe initial ayant la même valeur que le login doit être automatiquement fourni à l'utilisateur en attendant qu'il puisse le modifier au moyen de la méthode permettant de modifier le mot de passe. Cette méthode doit générer une exception de type *MotDePasseInvalideException* (à définir bien sûr) si le mot de passe fourni par l'utilisateur ne respecte pas la règle de définition des mots de passe.

Écrire une classe *UserManager* qui crée d'abord un objet utilisateur dont les informations sont saisies au moyen du composant Swing *JOptionPane* avant de lui faire changer son mot de passe.

Dire pourquoi les méthodes *get* et *set* doivent être publiques.

#### **Exercice 10 : Gestion des notes d'un étudiant.**

Pour la gestion des notes d'un étudiant, on se propose d'écrire les classes suivantes :

⇒ Une classe **Email** permettant de représenter une adresse email comme suite :  
nomUtilisateur@nomServeur, cette classe a :

- deux attributs privés nomUtilisateur et nomServeur.
- un constructeur **public Email (String nomUtilisateur, String nomServeur).**
- des accesseurs c'est à dire les méthodes get et set pour les attributs nomUtilisateur et nomServeur.
- une méthode **String toString()** permettant d'afficher une adresse email sous la forme : nomUtilisateur@nomServeur.

⇒ Une classe **InvalideSexeException** permettant de définir une exception java qui sera lancé quand on donne un sexe différent de M ou F.

⇒ Une classe **Personne** qui a :

- des attributs privés nom, sexe, téléphone et email de type **Email** précédemment déclaré.
- un constructeur public **Personne (String nom, char sexe, String tel, Email email) throws InvalideSexeException.** Cette classe lance une l'exception **InvalideSexeException** si le sexe en argument est différent des caractères **M** ou **F**.
- des accesseurs pour les attributs nom, sexe, téléphone et email
- une méthode **String toString()** qui permet d'afficher une personne sous le format : [Mr Mouhamed, 77 523 20 22, mouhamed@yahoo.fr] si c'est une personne de sexe M ou [Mme Fatima, 76 320 55 99, fatima@hotmail.com] si c'est une personne de sexe F.

⇒ Une classe **Etudiant** étendant la classe **Personne** qui a :

- des attributs crédit et notes qui est un tableau contenant les notes de l'étudiant.
- un constructeur sans argument.
- des accesseurs pour les attributs crédit et notes
- une méthode **calculerMoyenne().**
- une méthode **String toString()** qui permet d'afficher un étudiant sous le format : [Mr Mouhamed, 77 523 20 22, [mouhamed@yahoo.fr](mailto:mouhamed@yahoo.fr), 15 crédits et moyenne 17,5] si c'est un étudiant de sexe M ou [Mme Fatima, 76 320 55 99, fatima@hotmail.com, 15 crédits et moyenne 18,5] si c'est un étudiant de sexe F.

**Question : Écrire les classes Email, InvalideSexeException, Personne et Etudiant.**

### **Exercice 11: L'arbre généalogique**

On désire réaliser un système permettant de construire et d'afficher un arbre généalogique.

**Question a**

- a) Créer la classe Date qui contient une date sous la forme de 3 entiers, le jour, le mois et l'année.
- b) Définir un constructeur de la forme Date(int j, int m, int a) qui sert à construire une telle date.
- c) Définir une méthode versChaine() qui retourne une chaîne de caractère, représentation de la date sous la forme *jour/mois/année*.

Exemple:

```
Date d = new Date(20, 10, 1999);
```

```
System.out.println(d.versChaine()); // affiche: 20/10/1999
```

### Question B

- a) Définir la classe Personne qui est telle que toute personne est définie par son nom, son prénom, sa date de naissance et éventuellement par son père, sa mère (qui sont tous deux des personnes) et par sa date de décès.
- b) Faire un constructeur qui permet de créer une personne à partir de son nom, son prénom, le jour, le mois et l'année de sa date de naissance.
- c) Faire des méthodes d'accès pour la définition du pere, de la mere et de l'année de décès de la personne. Voilà ce que cela donne:

```
Personne jean = new Personne("Dupond", "Jean", 3, 1, 1966);
Personne marie = new Personne("Durand", "Marie", 5, 9, 1942);
Personne marcel = new Personne("Dupond", "Marcel", 15, 10, 1938);
Personne helene = new Personne("Leblanc", "Hélène", 21, 11, 1908);
Personne paul = new Personne("Durand", "Paul", 12, 06, 1905);
helene.deces(10,08,1986);
paul.deces(1,12,1970);
jean.pere(marcel); // Jean a comme pere Marcel
jean.mere(Marie);
marie.pere(Paul);
marie.mere(Helene);
```

- d) Faire une méthode afficher() dans Personne qui donne le résultat suivant:

**jean.afficher()**

*Jean Dupond*

*né(e) le 3/1/1996*

**helene.afficher()**

*Hélène Leblanc*

*né(e) le 21/11/1908*

*décédé(e) le 10/08/1986*

- e) Faire une méthode afficherGenealogie() dans Personne affiche toute la généalogie d'une personne. Exemple:

**jean.afficherGenealogie();**

*Jean Dupond*

*né(e) le 3/1/1996*

***a comme père :***

*Marcel Dupond*

*né(e) le 15/10/1938*

***a comme mère :***

*Marie Durand*

*né(e) le 5/9/1942*

***a comme père :***

*Paul Durand*

*né(e) le 5/9/1942*

*décédé(e) le 1/12/1970*

***a comme mère :***

*Hélène Leblanc*

*né(e) le 21/11/1908*

*décédé(e) le 10/08/1986*

**Note:** dans un premier temps on ne s'intéressera pas à l'indentation. Mais dans un deuxième temps on se posera la question de l'indentation. Idée: créer une méthode auxiliaire (que l'on appellera `afficherGenealogie1(int i)` où `i` est le nombre de blanc qui sont affichées avant les messages de sortie, et créer de même une méthode auxiliaire `afficher1(int i)` qui fait comme `afficher` mais qui place `i` blanc avant le message de sortie.